Week 10 Meeting 1: Quantum Phase Estimation

Student Lab Handout

Duration: 1h15

Learning Objectives

- Explain what Quantum Phase Estimation (QPE) computes
- Understand how eigenvalues are encoded into measurable phases
- Construct a QPE circuit step by step
- Extract energies from measurement data
- Compare QPE and VQE conceptually and practically
- Explain why QPE is not feasible on NISQ-era hardware

Timed Agenda (75 minutes)

1. **0:00-0:10**: Warm-up and instructions
2. **0:10-0:30**: QPE Circuit Logic (Concept $\rightarrow$ Circuit)
3. **0:30-0:50**: Implementing QPE for a One-Qubit Hamiltonian
4. **0:50-1:05**: Eigenstate vs Superposition Inputs
5. **1:10-1:15**: Wrap-up questions

1 Introduction

Eigenstates and Time Evolution For a time-independent Hamiltonian $\hat{H}$, quantum dynamics are governed by the time-evolution operator

$$\hat{U}(t) = e^{-i\hat{H}t}.$$

If the system is prepared in an *energy eigenstate*

$$\hat{H}\,|E\rangle = E\,|E\rangle\,,$$

then time evolution yields

$$\hat{U}(t)\,|E\rangle = e^{-iEt}\,|E\rangle\,.$$

Key idea: the energy eigenvalue E appears purely as a *phase*. Quantum Phase Estimation is precisely the algorithm that measures this phase.

What Does Quantum Phase Estimation Do? QPE estimates the phase ϕ associated with a unitary operator $\hat{U}$ such that

$$\hat{U}\,|\psi\rangle = e^{2\pi i\phi}\,|\psi\rangle\,.$$

When $\hat{U} = e^{-i\hat{H}t}$, the phase is directly related to energy:

$$\phi = -\frac{Et}{2\pi} \pmod 1.$$

Thus, measuring ϕ allows us to extract E.

2 QPE Circuit Logic

The QPE algorithm uses two registers:

- **Counting register:** n qubits that store the binary expansion of ϕ
- **System register:** qubits prepared in an eigenstate of $\hat{H}$

The algorithm proceeds as:

1. Prepare counting qubits in superposition
2. Apply controlled-$\hat{U}^{2^k}$ gates
3. Apply the inverse Quantum Fourier Transform (QFT)
4. Measure the counting register

Important: Without the inverse QFT, measurements appear as random noise.

3 Example: One-Qubit Hamiltonian

To isolate the algorithmic ideas behind Quantum Phase Estimation (QPE), we begin with the simplest nontrivial quantum system: a single qubit.

We consider the Hamiltonian

$$\hat{H} = \frac{\omega}{2}\,\sigma_z,$$

where σ_z is the Pauli-Z operator. Its action on the computational basis states is

$$\sigma_z\,|0\rangle = +\,|0\rangle\,, \qquad \sigma_z\,|1\rangle = -\,|1\rangle\,,$$

so that the corresponding energy eigenvalues are

$$E_\pm = \pm\frac{\omega}{2}.$$

3.1 Time-Evolution Operator

For a time-independent Hamiltonian, the time-evolution operator is

$$\hat{U}(t) = e^{-i\hat{H}t}.$$

Substituting the Hamiltonian above, we obtain

$$\hat{U}(t) = e^{-i(\omega t/2)\sigma_z} = \begin{pmatrix} e^{-i\omega t/2} & 0 \\ 0 & e^{+i\omega t/2} \end{pmatrix}.$$

Acting on the energy eigenstates,

$$\hat{U}(t)\,|0\rangle = e^{-i\omega t/2}\,|0\rangle\,, \qquad \hat{U}(t)\,|1\rangle = e^{+i\omega t/2}\,|1\rangle\,.$$

Key observation: time evolution leaves each eigenstate unchanged except for a phase factor that depends on its energy.

3.2 Energy as a Phase

To connect this result with Quantum Phase Estimation, we compare it with the generic eigenvalue equation for a unitary operator,

$$\hat{U}\,|\psi\rangle = e^{2\pi i\phi}\,|\psi\rangle\,.$$

Identifying the phase factors, we find that the energy E is encoded in the phase ϕ according to

$$\phi = -\frac{Et}{2\pi} \pmod 1.$$

Here (mod 1) indicates that phases are defined only up to an integer, so only the fractional part of ϕ is physically meaningful.
This relation captures the central idea of Quantum Phase Estimation: instead of measuring the energy directly, the algorithm determines it by measuring the phase accumulated under controlled time evolution. Once the phase ϕ is known, the energy E can be reconstructed.

For simplicity, we choose the evolution time

$$t = 2\pi.$$

With this choice, the unitary

$$\hat{U} = e^{-i\hat{H}t}$$

acts as a simple phase-rotation operator on the computational basis states. This minimal setup avoids additional mapping overhead, allows immediate comparison with the exact eigenvalues, and makes transparent how Quantum Phase Estimation extracts spectral information from controlled time evolution.

4 Step-by-Step QPE Implementation

```
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFT
from qiskit.quantum_info import Operator, Statevector
from qiskit_ibm_runtime import SamplerV2 as Sampler

import numpy as np
import matplotlib.pyplot as plt
```

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.linalg import expm          # Use scipy for matrix exponential
from qiskit.quantum_info import Operator
```

```
# Define constants
omega = 1.0
t = 2 * np.pi  # Convenient choice

# Pauli-Z Hamiltonian
Z = np.array([[1, 0],
              [0, -1]])

H = (omega / 2) * Z

# Time evolution operator
U = Operator(expm(-1j * H * t))
```

```
# Eigenvalues and eigenvectors (for reference)
eigvals, eigvecs = np.linalg.eigh(H)
eigvals, eigvecs
```

Registers

- $n = 3$ counting qubits (phase precision 2^{-3} - the phase is stored with a resolution of 1/8, so the algorithm can only distinguish phases that differ by at least 2^{-3}.)
- 1 system qubit

```
num_counting_qubits = 3
# Number of qubits used to store the phase.
# This gives a phase precision of 1 / 2^3 = 1/8,
# meaning QPE can resolve the phase only up to multiples of 1/8.

num_system_qubits = 1
```

```
qc = QuantumCircuit(num_counting_qubits + num_system_qubits)

counting = list(range(num_counting_qubits))
# Indices of the counting (control) qubits.
# These qubits store the binary digits of the phase $\phi$ measured by QPE.
# For num_counting_qubits = 3, this corresponds to qubits [0, 1, 2].

system = [num_counting_qubits]
# Index of the system (target) qubit.
# This qubit holds the eigenstate of the Hamiltonian and is acted on by U.
# It comes immediately after the counting register in the circuit.

print(counting, system)
```

State Preparation

If the system register is prepared in a single energy eigenstate, Quantum Phase Estimation returns a single phase value and therefore a single energy eigenvalue. In contrast, if the system is prepared in a superposition, for example

$$|\psi\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle + |1\rangle\big),$$

the state contains contributions from both energy eigenstates. In this case, QPE projects onto an eigenstate during the measurement and returns the corresponding energy eigenvalue, with probabilities determined by the overlaps of the initial state with each eigenstate.

```
qc.x(system[0])  # |1> is an eigenstate of Z
#qc.h(system[0]) # Prepares (|0> + |1>) / sqrt(2)
```

Controlled Unitaries

Each counting qubit controls a power of the time-evolution operator,

$$\hat{U}^{2^k},$$

where k labels the position of the qubit in the counting register. These controlled operations cause the phase accumulated by the system qubit to be "kicked back" onto the counting register. As a result, the binary digits of the phase are encoded across the counting qubits and can be extracted by the inverse Quantum Fourier Transform.

Why do we use 2^k?

The reason is that Quantum Phase Estimation (QPE) stores the phase ϕ in binary form. The qubit with index k is responsible for the k-th binary digit of the phase. To extract that digit, the system must evolve for a time proportional to 2^k. That is why the controlled operation is $\hat{U}^{2^k}$.

Example (with $n = 3$)

Counting Qubit	Index k	Applied Unitary
qubit 0	$k = 0$	$\hat{U}^1$
qubit 1	$k = 1$	$\hat{U}^2$
qubit 2	$k = 2$	$\hat{U}^4$

Each qubit accumulates twice as much phase as the previous one.

You can think of this as letting the system evolve for different lengths of time:

- $k = 0$: one unit of time $\rightarrow$ coarse phase information
- $k = 1$: twice as long $\rightarrow$ finer information
- $k = 2$: four times as long $\rightarrow$ even finer information

Together, these controlled evolutions encode the phase ϕ digit by digit into the counting register. Here k labels the counting qubits, and the controlled operation $\hat{U}^{2^k}$ ensures that each qubit records a different binary digit of the phase.

```
qc.h(counting)
# Apply a Hadamard gate to each counting qubit.
# This prepares the counting register in an equal superposition of all
    computational
# basis states, which is necessary for encoding the phase information.
# Without this superposition, QPE would not be able to extract the phase.
```

```
from qiskit.circuit.library import UnitaryGate

U_gate = UnitaryGate(U)
for i, qubit in enumerate(counting):
    power = 2 ** i
    controlled_U = U_gate.power(power).control()
    qc.append(controlled_U, [qubit] + system)
```

Inverse QFT

The inverse QFT converts relative phases into computational basis amplitudes, enabling direct measurement.

```
from qiskit.circuit.library import QFTGate

# The .inverse() method is used to get the Inverse QFT
qft_inv_gate = QFTGate(num_counting_qubits).inverse()
qc.append(qft_inv_gate, counting)
```

```
qc.measure_all()
qc.draw('mpl')
```

```
from qiskit.primitives import StatevectorSampler

sampler = StatevectorSampler()
job = sampler.run([qc], shots=2048)
result = job.result()
pub_result = result[0]
counts = pub_result.data.meas.get_counts()
```

5 Extracting the Energy

After measurement, we obtain a bitstring

$$b = b_1 b_2 \dots b_n,$$

corresponding to the phase

$$\phi = \frac{b}{2^n}.$$

For $t = 2\pi$,

$$E = -\phi \pmod 1.$$

Care must be taken to unwrap the phase and map it into the physical energy range.

```
plt.bar(counts.keys(), counts.values())
plt.xlabel("Measured Bitstring")
plt.ylabel("Probability")
plt.title("QPE Measurement Results")
plt.show()
```

```
# Energy Estimation Logic / only for initial |1>
bitstring = max(counts, key=counts.get)
phase_decimal = int(bitstring, 2) / (2 ** num_counting_qubits)

# Mapping phase to Energy: E = -(2 * pi * phase) / t
# We use the negative because U = exp(-iHt)
estimated_energy = -(2 * np.pi * phase_decimal) / t

# Handle periodicity (phases are mod 1, energies are mod 2pi/t)
if estimated_energy <= -1.0:
    estimated_energy += 1.0

print(f"Most Probable Bitstring: {bitstring}")
print(f"Exact eigenvalues: {eigvals}")
print(f"Estimated energy from QPE: {estimated_energy}")
```

```
print("Exact eigenvalues:", eigvals)
print("Estimated energy from QPE:", estimated_energy)
```

6 Results and Interpretation

Simulation results show:

- Sharp peaks at bitstrings corresponding to exact eigenvalues
- Increasing n improves phase precision
- Superposition inputs reveal multiple eigenvalues

Conclusion: Within finite phase resolution, QPE returns *exact* eigenvalues.

7 Exercise

Repeat the experiment using an initial state that is a superposition of both eigenstates. What do you observe?

8 Why QPE Is Not NISQ-Friendly

QPE is currently impractical because:

- Controlled unitaries are costly and noisy
- Circuit depth scales with desired precision
- Long coherence times are required

9 Reflection Questions

1. Why must the system register contain an eigenstate?
2. How does the number of counting qubits affect precision?
3. What role does the inverse QFT play?
4. Why do controlled unitaries limit NISQ feasibility?